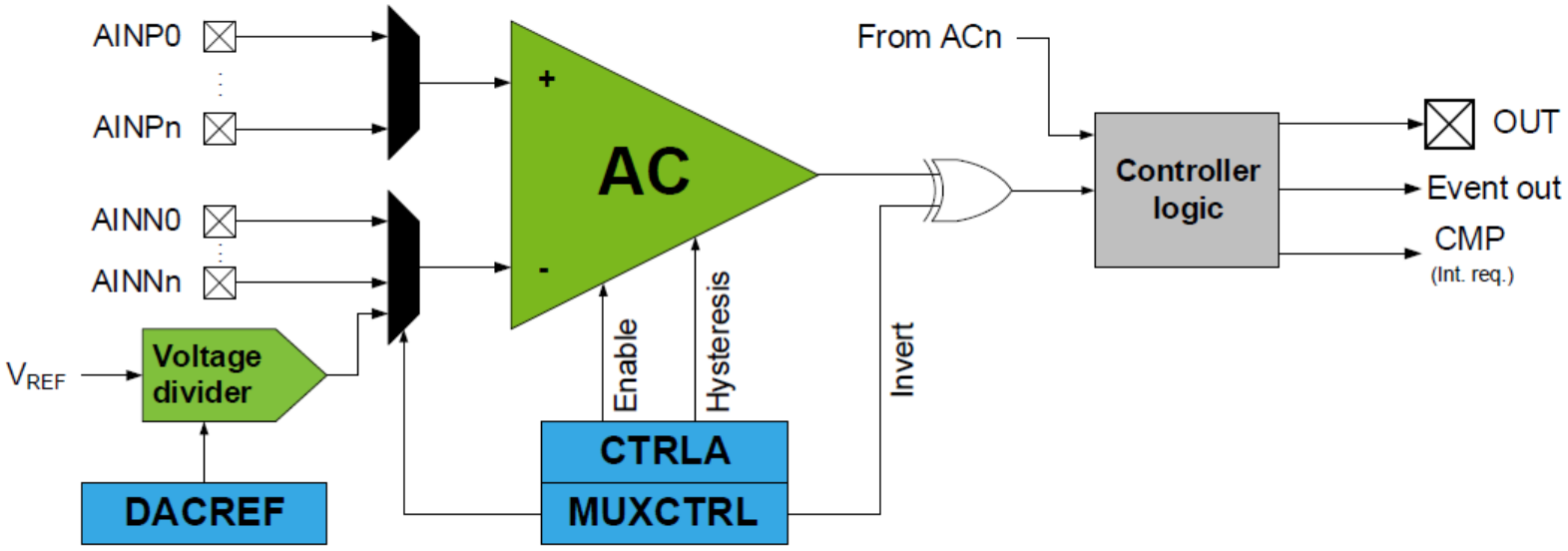
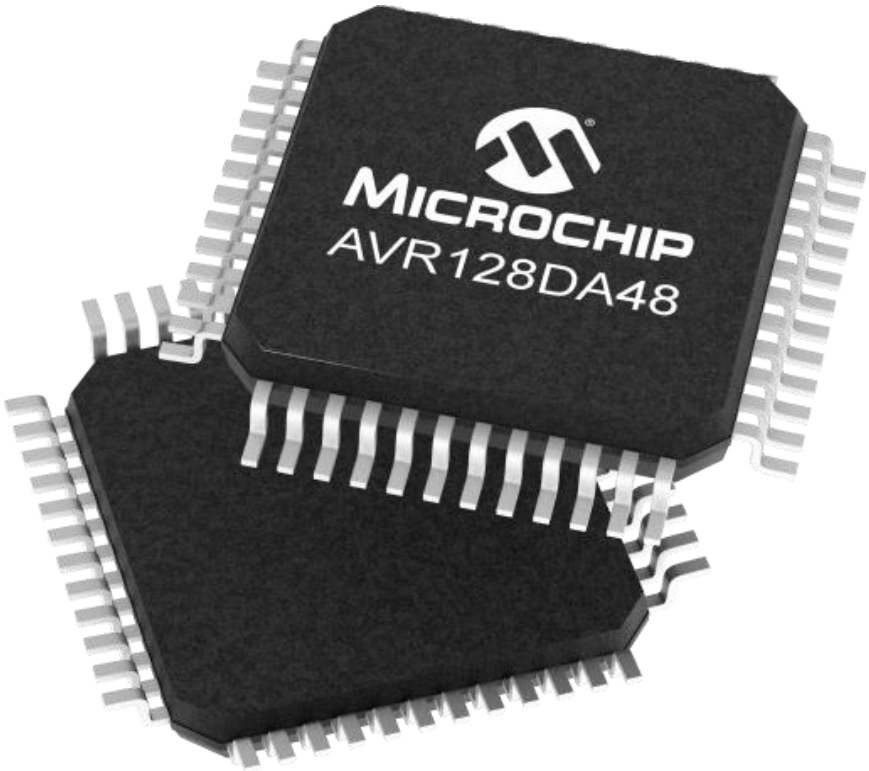
“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 20 - Analog Comparator (AC)





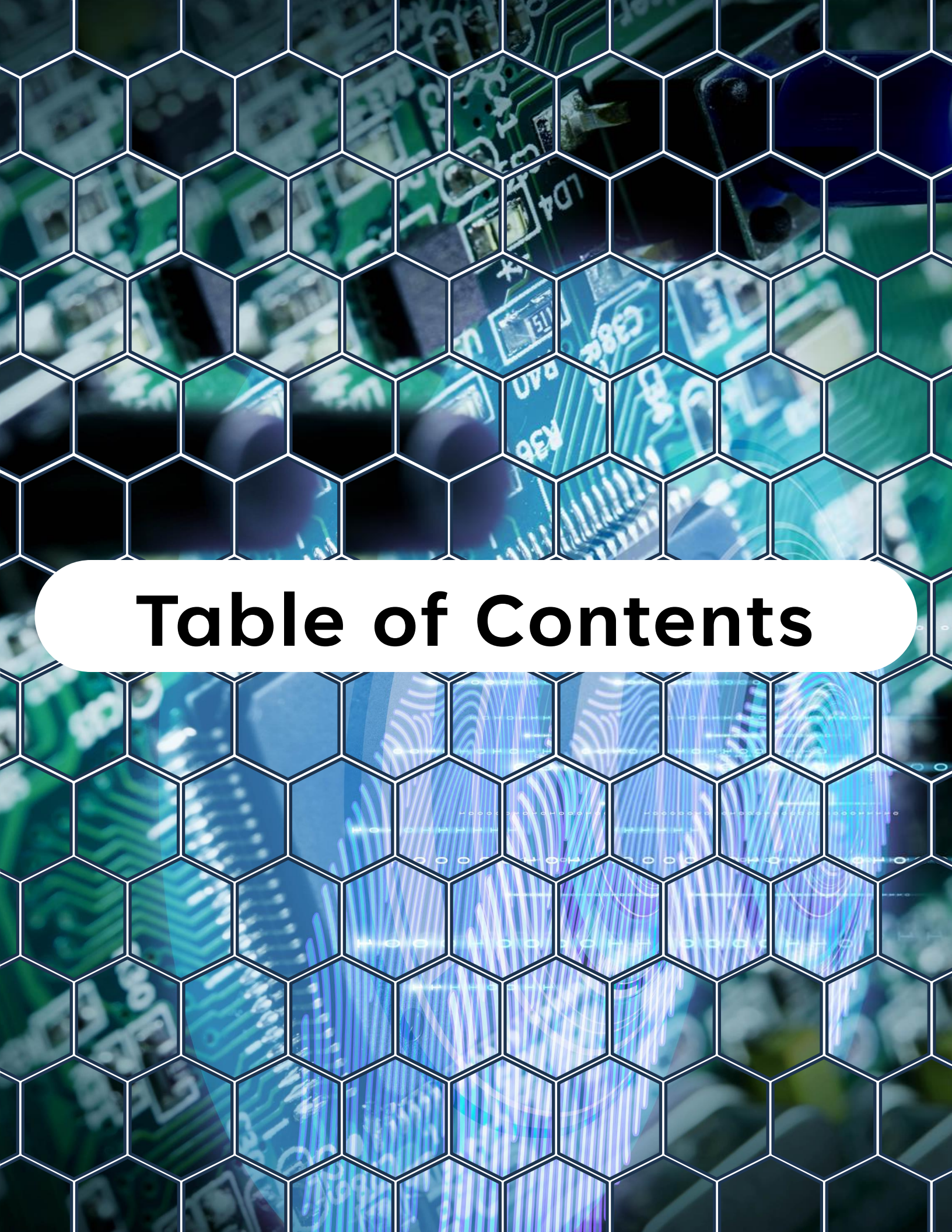
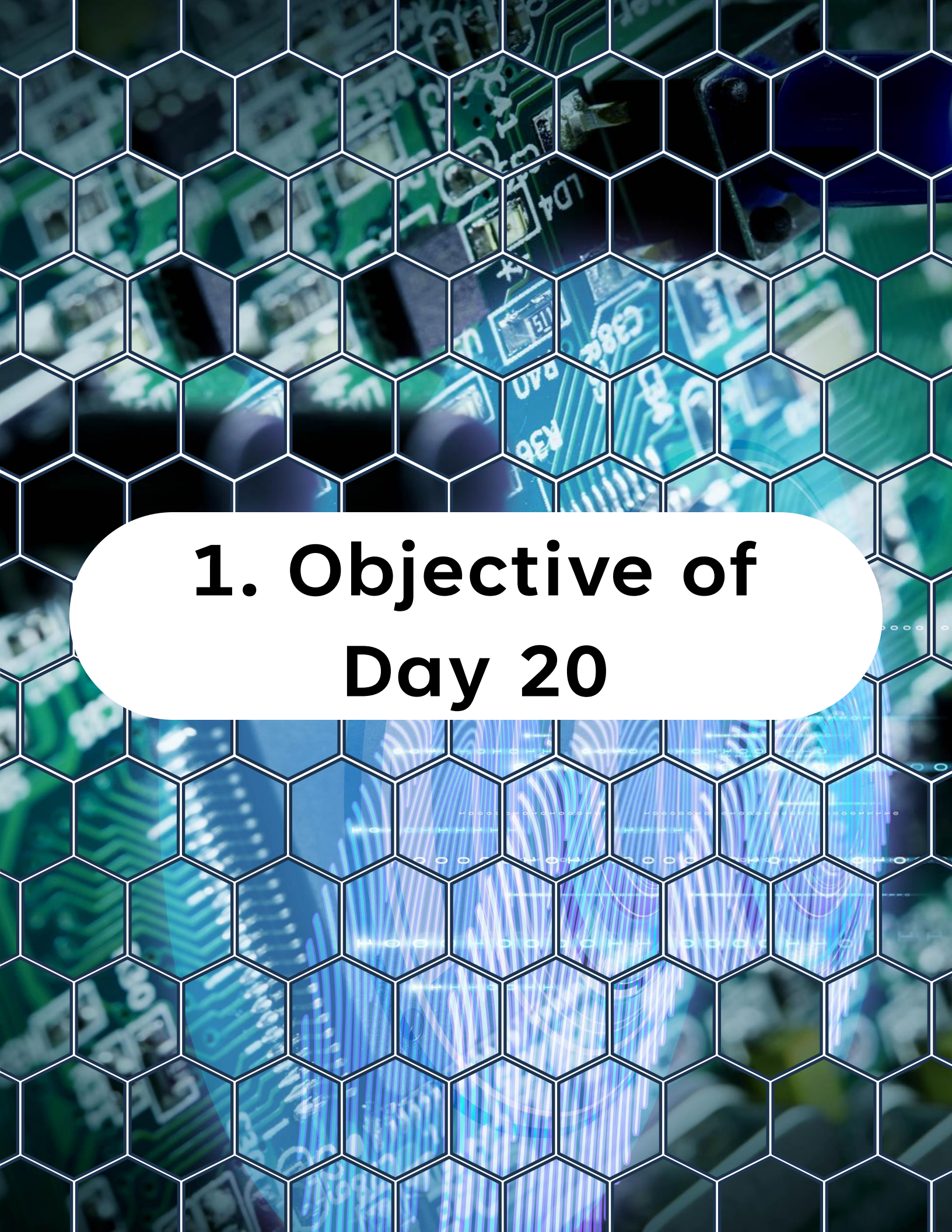


Table of Contents

Table of Contents



1. Objective of Day 20

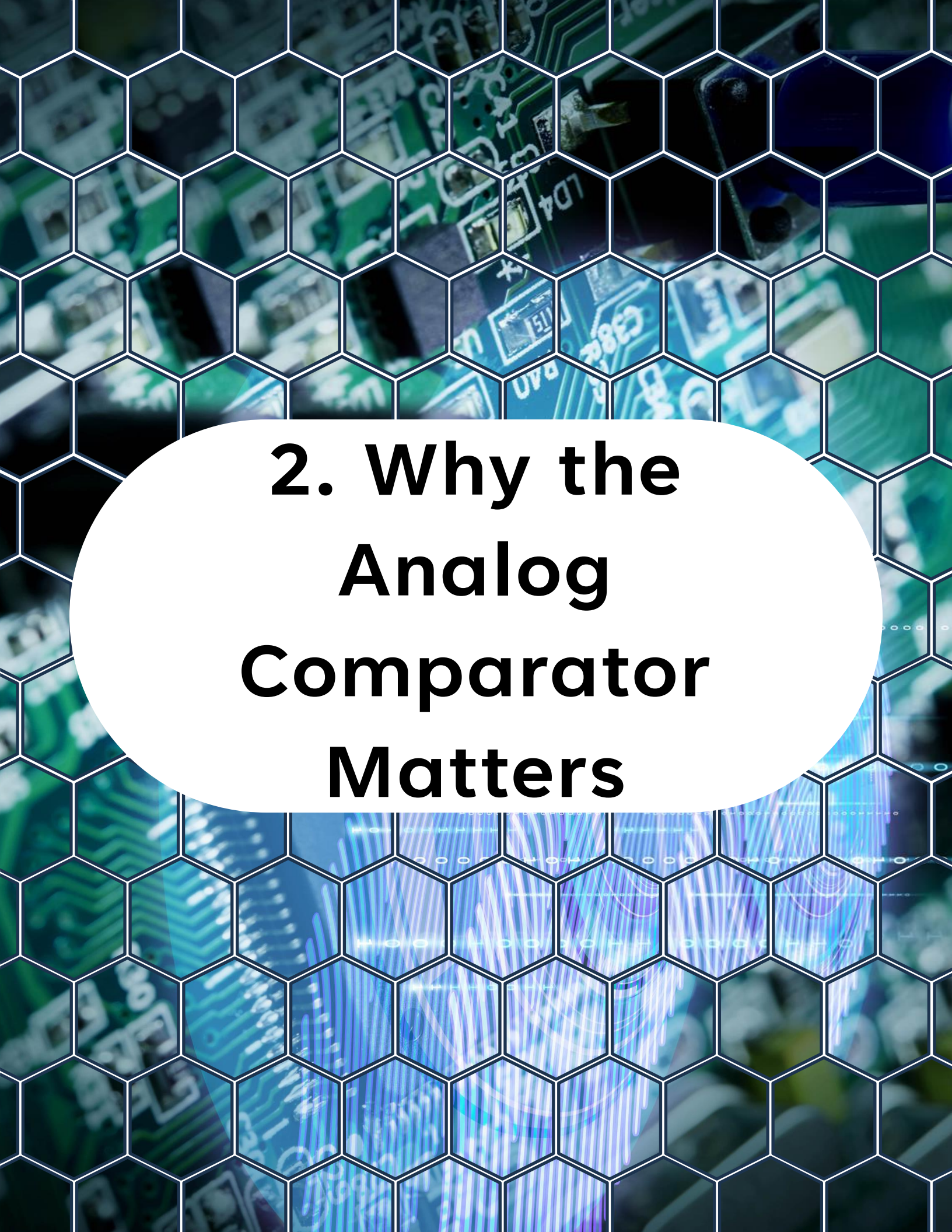
1. Objective of Day 20

Today is where the AVR DA family starts to feel "modern".

You will learn how to:

- Configure the **Analog Comparator (AC)**
- Create clean digital events from analog signals (threshold crossings)
- Route **AC** outputs through the Event System (**EVSYS**)
- Trigger peripherals without CPU involvement
- Combine **AC** + **EVSYS** for low-latency, low-power designs

By the end of the day, you will be able to build systems where the CPU is not babysitting inputs.



2. Why the Analog Comparator Matters

2. Why the Analog Comparator Matters

ADC is for measuring.

Comparator (AC) is for deciding.

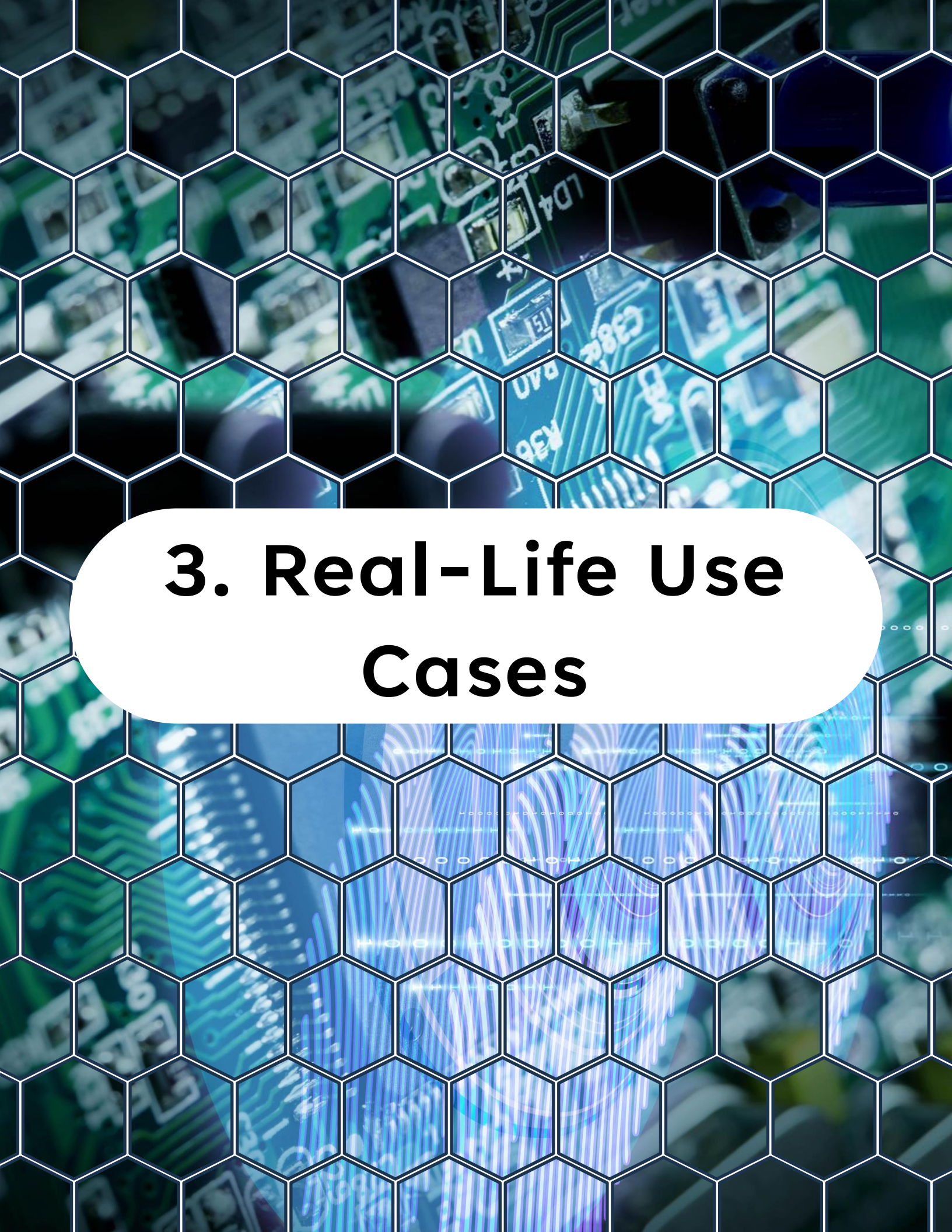
If your firmware needs to answer questions like:

- "Did voltage cross 2.8 V?“
- "Did current exceed limit?“
- "Is signal above a threshold right now?“
- "Did it just cross the threshold (edge)?“

...then the analog comparator is the right tool.

Key benefits:

- Very fast reaction (compared to polling an ADC)
- Can run while CPU sleeps
- Can generate interrupts or events

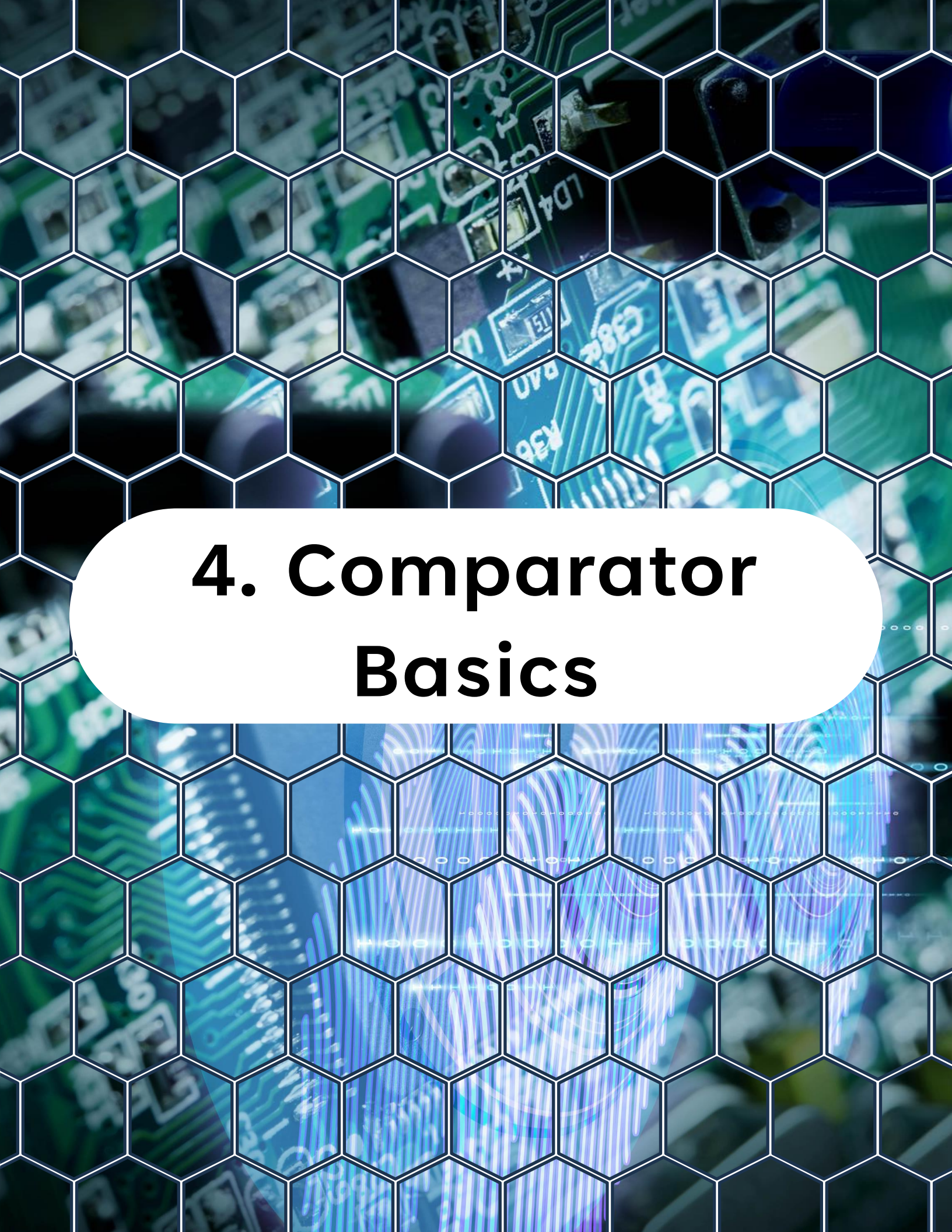


3. Real-Life Use Cases

3. Real-Life Use Cases

Here are common reasons to use AC on real boards:

- Brownout-like custom threshold (beyond built-in BOR)
- Zero-cross detection for AC waveform timing
- Overcurrent / overvoltage cutoffs
- Sensor threshold detection (light, pressure, battery)
- Schmitt-trigger style cleanup for noisy analog inputs
- Wake-on-threshold while sleeping



4. Comparator Basics

4. Comparator Basics

The **analog comparator (AC)** compares the voltage levels on two inputs and gives a digital output based on this comparison.

A comparator has two inputs:

- Positive input: **VIN+**
- Negative input: **VIN-**

Output:

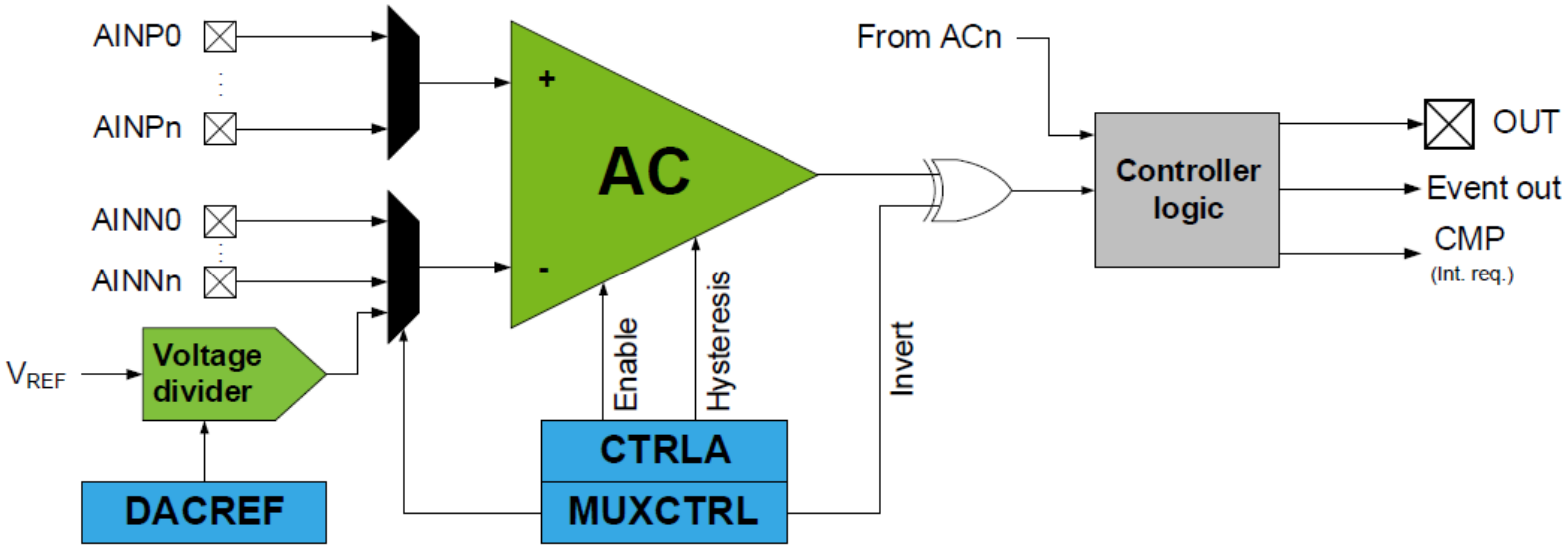
- 1 when **VIN+ > VIN-**
- 0 when **VIN+ < VIN-**

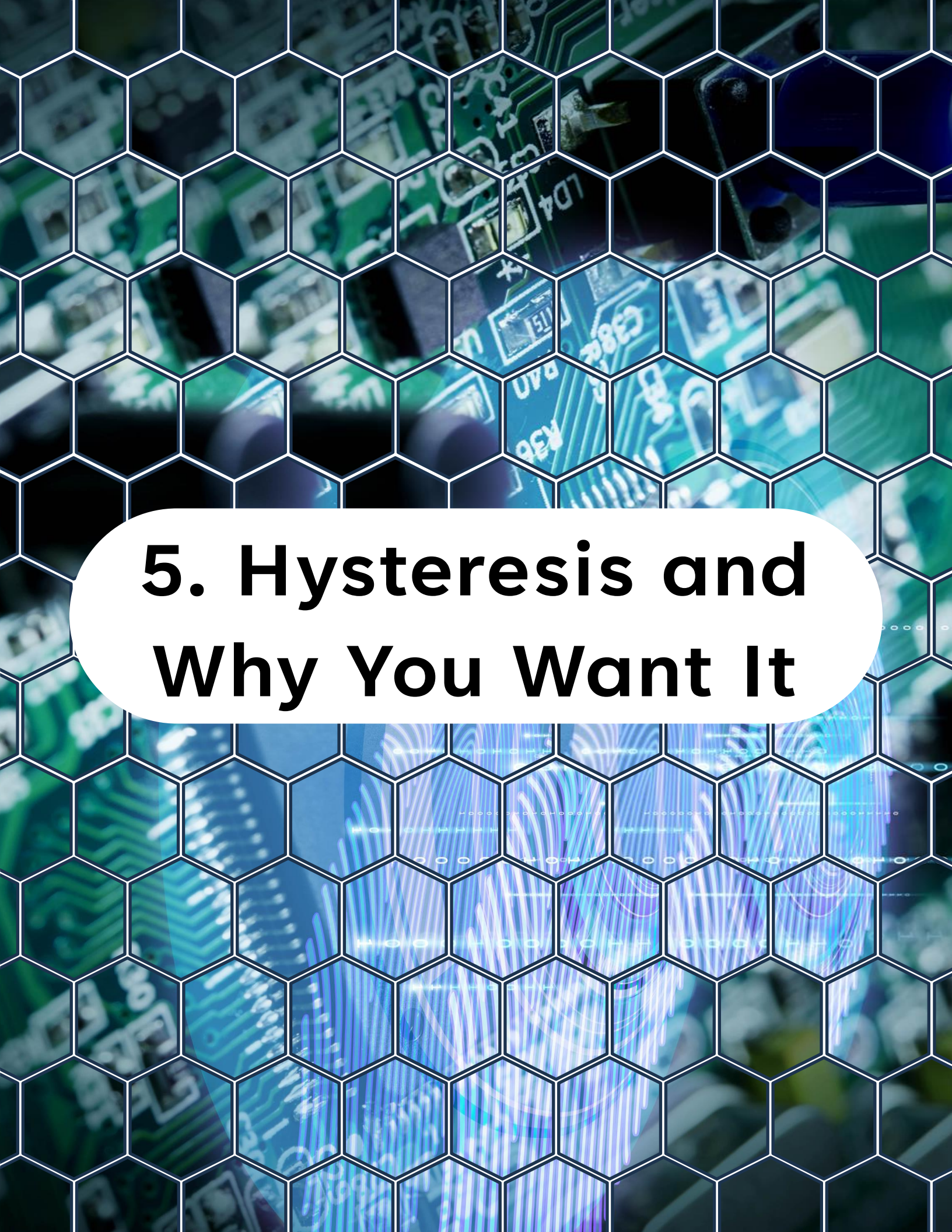
Usually you set:

- **VIN+** = external analog signal
- **VIN-** = reference voltage (**DAC**, **VREF**, bandgap, divider, pin)

4. Comparator Basics

Analog Comparator (AC) Block Diagram





5. Hysteresis and Why You Want It

5. Hysteresis and Why You Want It

Real signals are noisy.

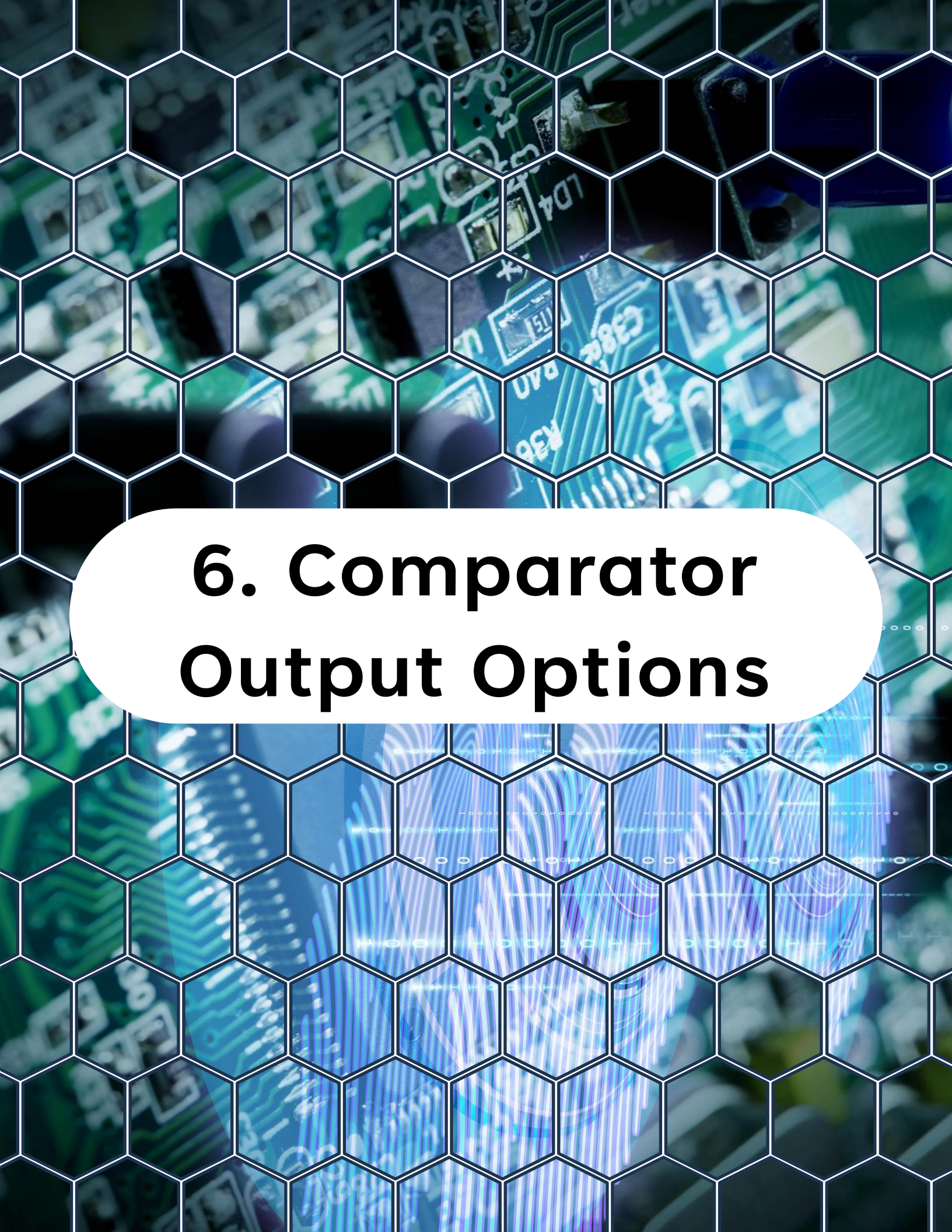
When the input hovers near the threshold, output can chatter.

Applying an input hysteresis helps to prevent constant toggling of the output when the noise-afflicted input signals are close to each other.

Hysteresis adds a small "dead band" so the threshold is slightly different when rising vs falling.

Rule of thumb:

- Noisy signals: enable hysteresis
- Clean signals: you can disable it for maximum sensitivity



6. Comparator Output Options

6. Comparator Output Options

On AVR DA, comparator output can be used in multiple ways:

- Polled (read a status bit)
- Interrupt-driven (ISR on rising/falling edge)
- Event-driven (**EVSYS** to other peripherals)

Interrupt-driven is fine, but **event-driven** is the big win when you want "hardware reacts to hardware".

7. What Is the Event System

7. What Is the Event System

The **Event System (EVSYS)** enables direct peripheral-to-peripheral signaling. It allows a change in one peripheral (the event generator) to trigger actions in other peripherals (the event users) through event channels, without using the CPU.

The Event System is basically an internal wiring matrix.

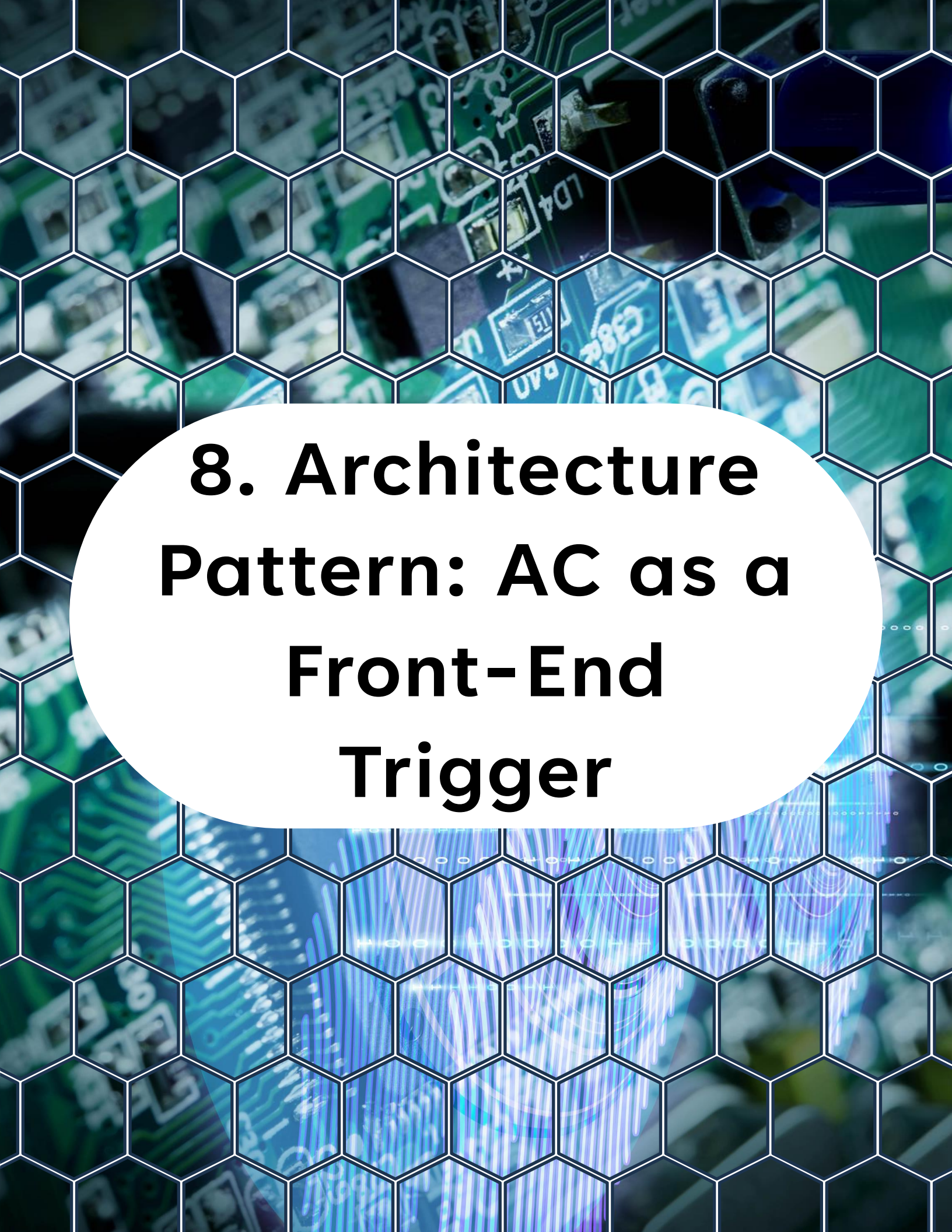
- Producers (**event generators**): AC, pins, timers, RTC, etc.
- Consumers (**event users**): timers, ADC triggers, CCL, etc.

7. What Is the Event System

Events move:

- Without CPU
- With low latency
- While sleeping

This is a core trick for building responsive systems with lower power and simpler firmware logic.


8. Architecture Pattern: AC as a Front-End Trigger

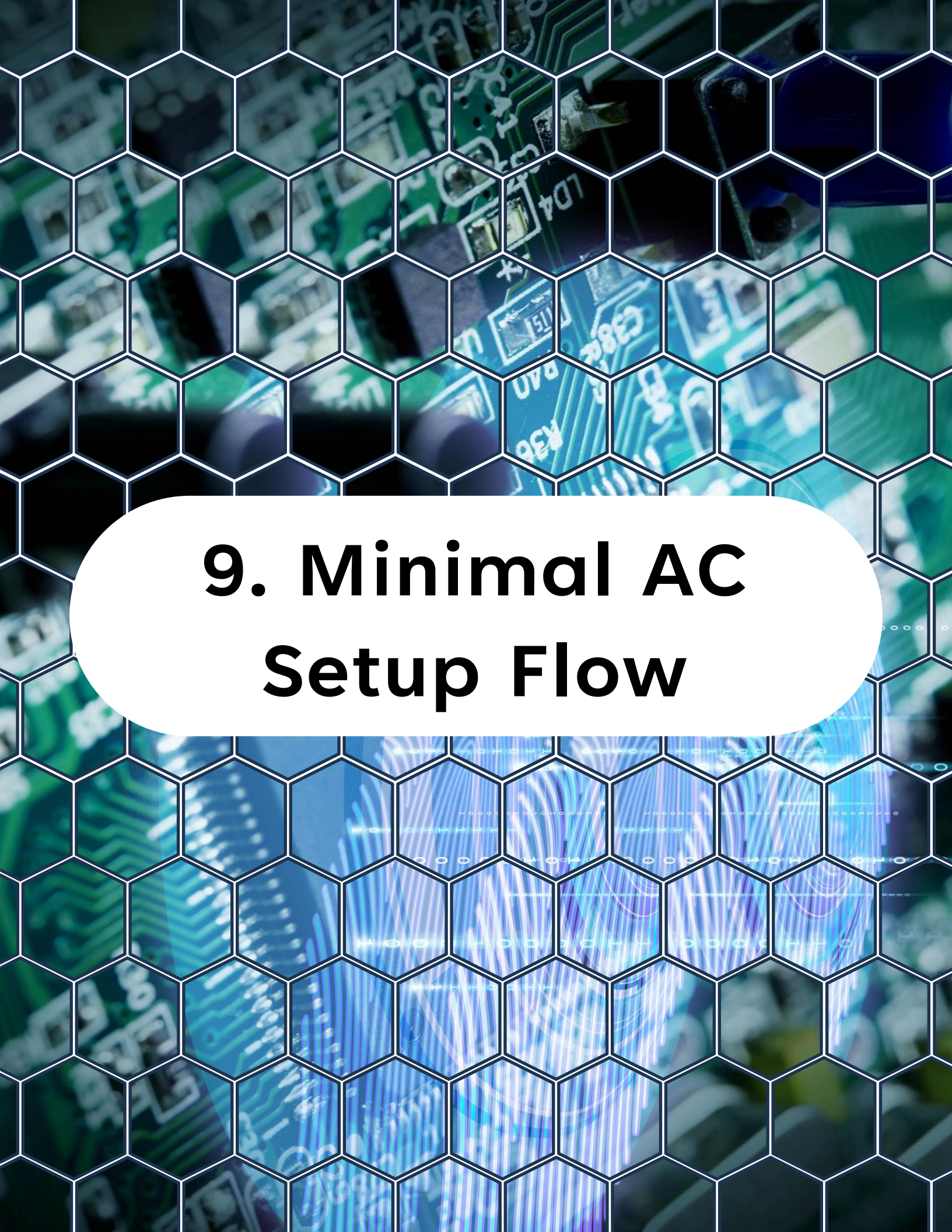
8. Architecture Pattern: AC as a Front-End Trigger

A clean pattern for many designs:

1. **AC** watches an analog signal and outputs a clean digital decision
2. **EVSYS** routes that decision to a timer or other peripheral
3. **Timer** captures timestamp or counts pulses
4. **CPU** reads results later (or wakes on interrupt)

Result:

- **CPU** is not constantly sampling or checking thresholds



9. Minimal AC Setup Flow

9. Minimal AC Setup Flow

Typical steps:

1. Select inputs (VIN+ source and VIN- source)
2. Set hysteresis (enable if needed)
3. Configure output polarity (optional)
4. Enable comparator
5. Choose action:
    - Interrupt
    - event output
    - Both

Exact register names vary by **AC** instance and header pack, so in the project examples you will map these through the device headers.

9. Minimal AC Setup Flow

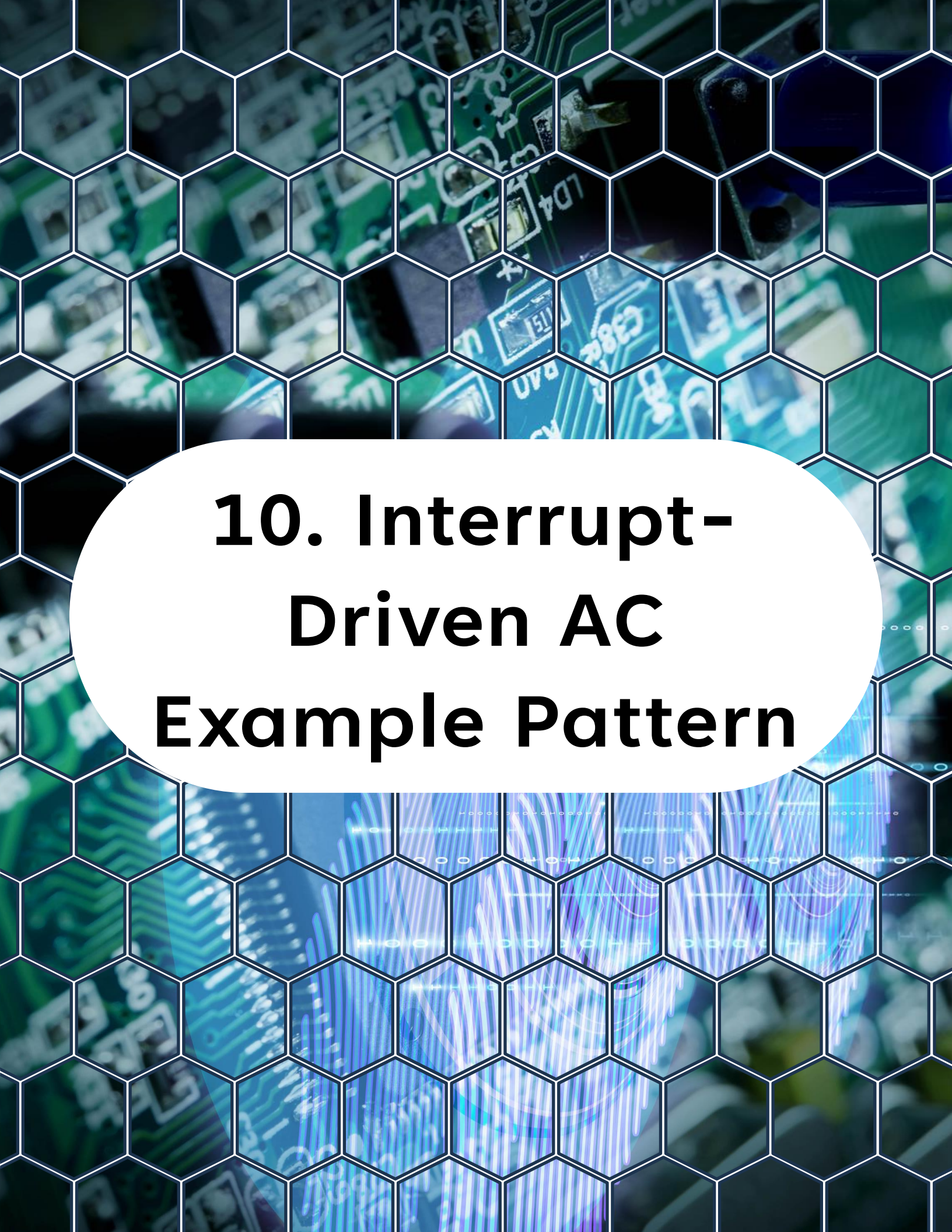
Microchip **AC** block diagrams show typical muxed inputs, hysteresis, enable, and interrupt paths.

Positive Input MUX Selection

Value	Name	Description
0x0	AINP0	Positive Pin 0
0x1	AINP1	Positive Pin 1
0x2	AINP2	Positive Pin 2
0x3	AINP3	Positive Pin 3
Other	-	Reserved

Negative Input MUX Selection

Value	Name	Description
0x0	AINN0	Negative Pin 0
0x1	AINN1	Negative Pin 1
0x2	AINN2	Negative Pin 2
0x3	DACREF	DAC Reference
Other	-	Reserved



10. Interrupt-Driven AC Example Pattern

10. Interrupt-Driven AC Example Pattern

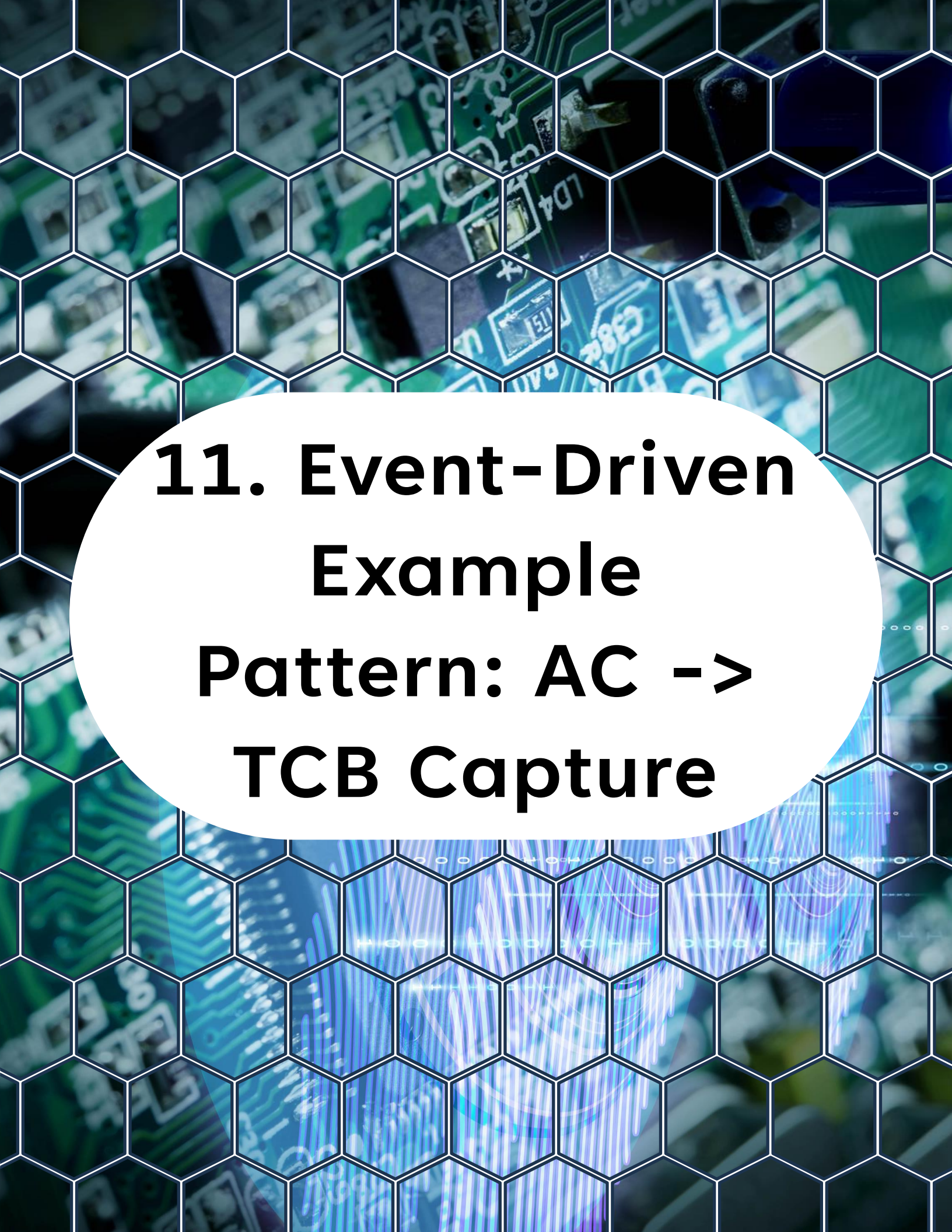
This pattern is good for learning and debugging:

- Trigger ISR on rising edge (threshold crossed upward)
- Toggle LED or timestamp event

Conceptually:

- Configure AC interrupt mode (rising/falling/both)
- Clear flags
- Enable IRQ
- In ISR: clear flag, do minimal work

Keep ISR short. Use it mainly to set flags or capture a timestamp.


11. Event-Driven Example
Pattern: AC -> TCB Capture

11. Event-Driven Example Pattern: AC -> TCB Capture

This is the kind of setup you use for:

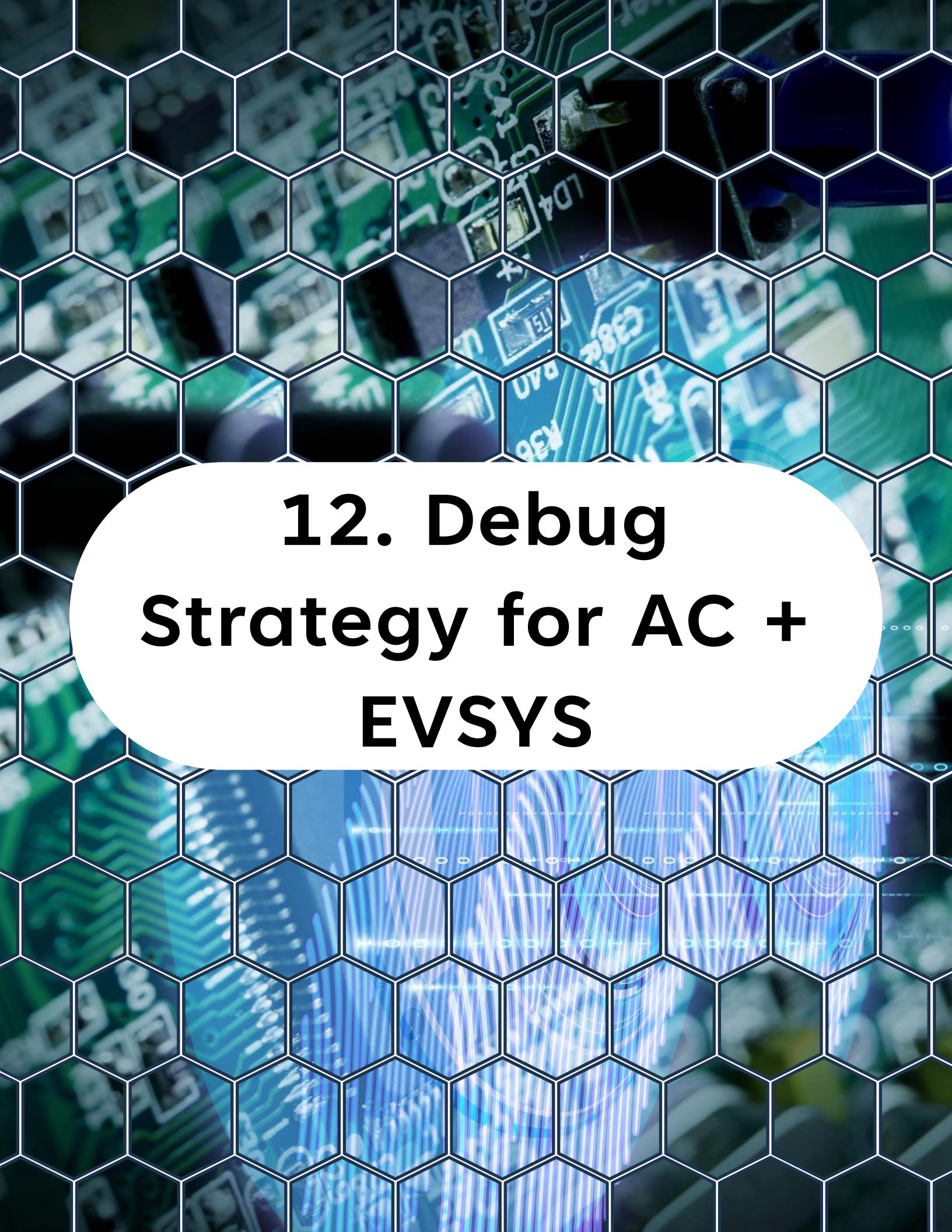
- measuring period between threshold crossings
- detecting frequency
- timing zero-crossings

Flow:

- AC output generates an event when it changes
- EVSYS routes that event to TCB capture input
- TCB captures timer count on event edges
- CPU reads captured values when convenient

This is cleaner than:

- polling pin state
- taking ADC samples in a loop



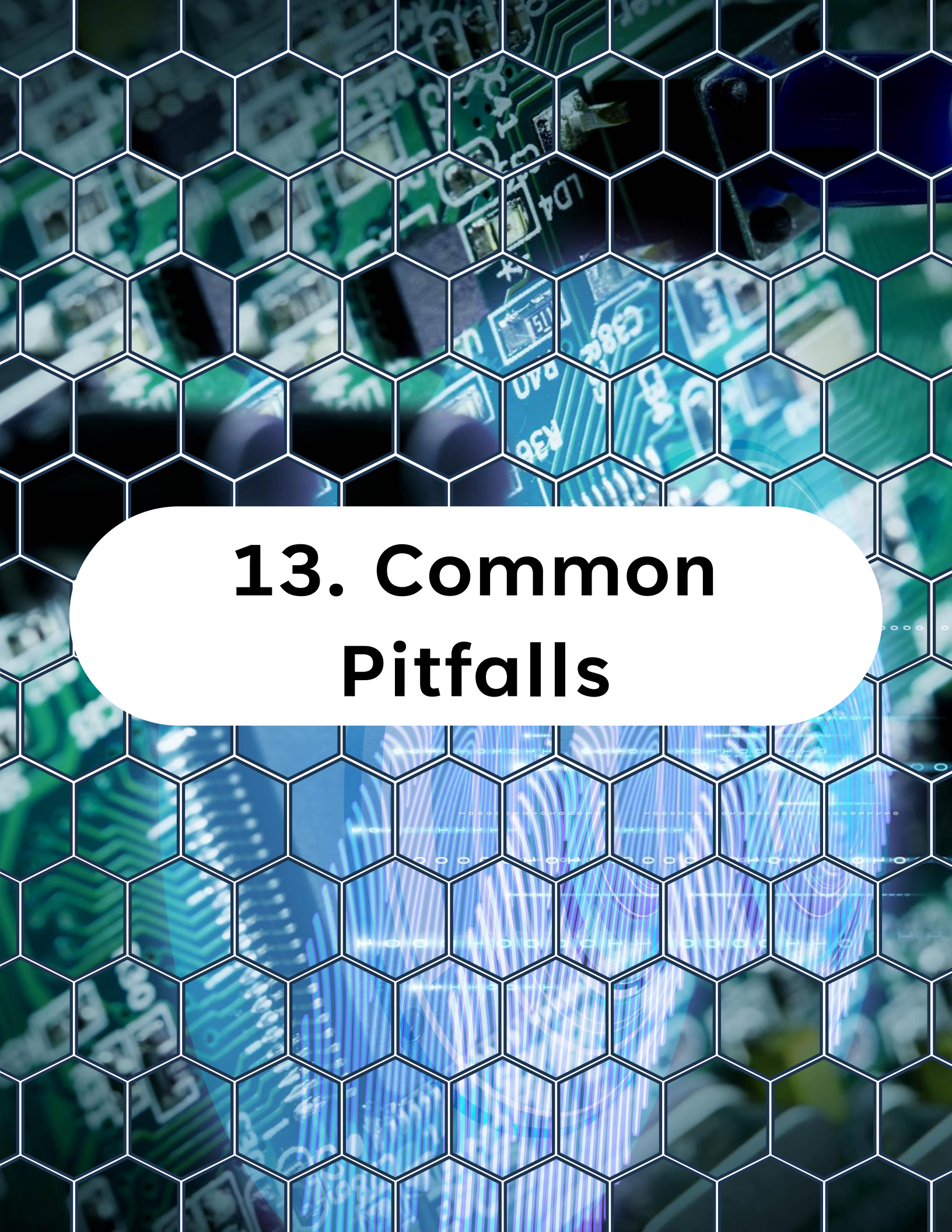
12. Debug Strategy for AC + EVSYS

12. Debug Strategy for AC + EVSYS

This combo is powerful, but debugging needs structure:

- Step 1: Verify AC alone (read status / use ISR toggle LED)
- Step 2: Verify EVSYS channel configured (route to something visible, like a timer toggle)
- Step 3: Verify consumer peripheral reacts (capture flag changes, count increments)
- Step 4: Only then integrate into app logic

If you try to debug **AC** + **EVSYS** + **timer** all at once, it gets messy fast.



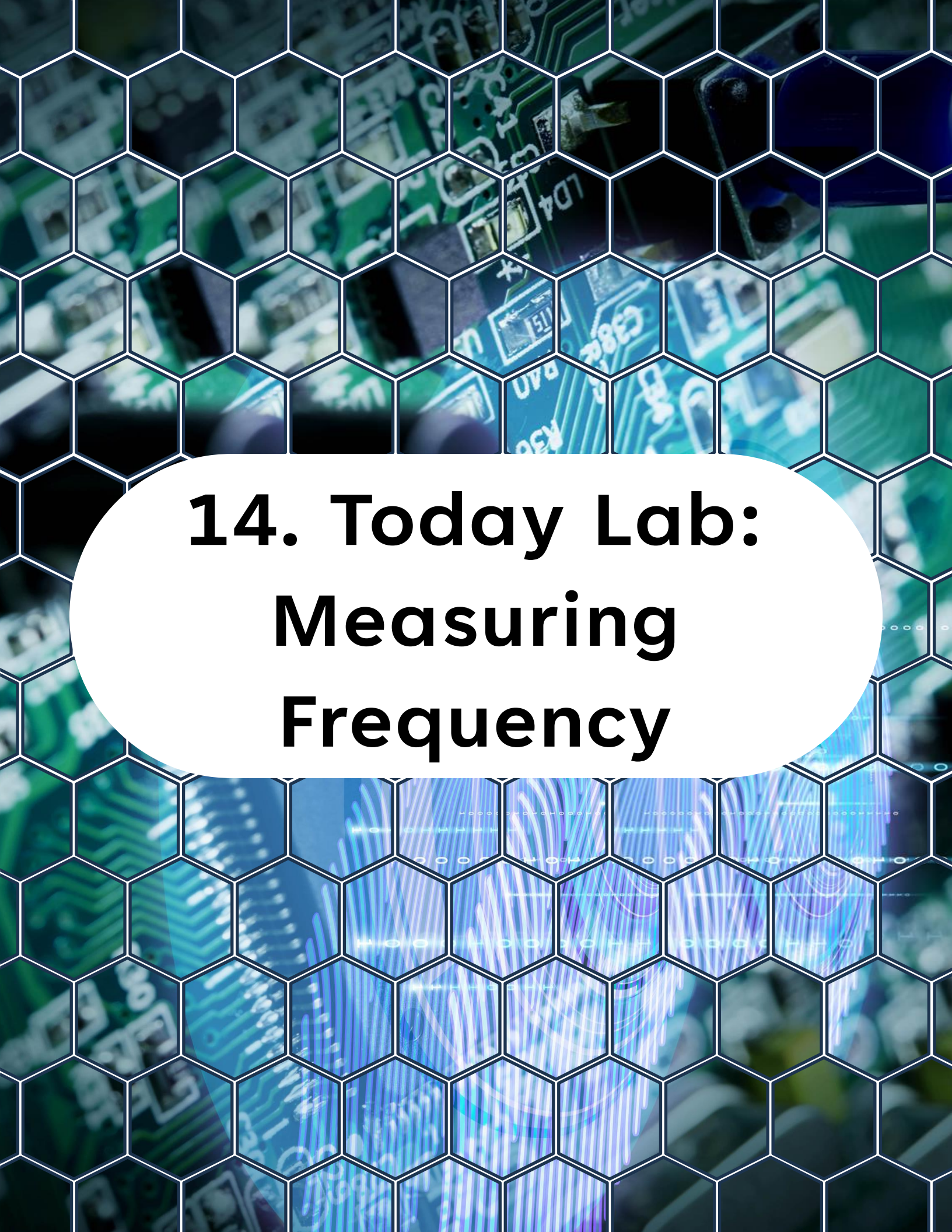
13. Common Pitfalls

13. Common Pitfalls

- Wrong input mux selection (**VIN+** and **VIN-** swapped)
- Reference not enabled (**VREF**/**DAC** not configured)
- No hysteresis on noisy signal (output chatters)
- Forgetting to clear interrupt flags
- Misconfigured event channel/user mapping
- Assuming pin routing when **PORTMUX** differs

Treat it like wiring on a PCB:

verify each connection.



14. Today Lab: Measuring Frequency

14. Today Lab: Measuring Frequency

This lab demonstrates PWM generation and hardware frequency measurement using TCA0, AC0, the Event System (EVSYS), and TCB0 — all without any software polling.

- **TCA0**: Generates a 50% duty cycle PWM signal on WO0 (PA0) at a known frequency.
- **AC0**: Detects the rising edge of the PWM signal fed into its positive input (PA7). Acts as an event generator via the Event System.
- **EVSYS**: Routes AC0 output events to TCB0 as the event user.
- **TCB0**: Configured in Frequency Measurement (FRQMEAS) mode.

14. Today Lab: Measuring Frequency

```c
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>

/* ---------------------------------------------------------------------------
 * Configuration constants
 * --------------------------------------------------------------------------- */

 #define F_CPU 4000000UL

/**
 * @brief TCA0 prescaler selection.
 * Using DIV16 so the counter runs at F_CPU/16 = 250 000 Hz.
 */
#define TCA0_PRESCALER_DIV      TCA_SINGLE_CLKSEL_DIV16_gc

/**
 * @brief TCA0 period register value (PER).
 *
 * Desired PWM frequency = 1 kHz.
 * Timer clock = F_CPU / 16 = 250 000 Hz.
 * PER = (timer_clock / pwm_freq) - 1 = (250000 / 1000) - 1 = 249.
 */
#define TCA0_PER_VALUE          249U

/**
 * @brief TCA0 compare register value for 50 % duty cycle.
 * CMP0 = (PER + 1) / 2 = 125.
 */
#define TCA0_CMP0_VALUE         125U

/**
 * @brief TCB0 prescaler selection.
 * Using DIV2 so the capture counter runs at F_CPU/2 = 2 000 000 Hz.
 */
#define TCB0_PRESCALER_DIV      TCB_CLKSEL_DIV2_gc

/**
 * @brief Clock frequency seen by TCB0 after prescaling (Hz).
 * Used to convert captured ticks to frequency.
```

14. Today Lab: Measuring Frequency

```
38  /**
39   * @brief Clock frequency seen by TCB0 after prescaling (Hz).
40   * Used to convert captured ticks to frequency.
41   */
42  #define TCB0_CLOCK_HZ           (F_CPU / 2UL)
43
44  /* ---------------------------------------------------------------------------
45   * Global variables
46   * --------------------------------------------------------------------------- */
47
48  /**
49   * @brief Most recently measured PWM frequency in Hz.
50   *
51   * Updated inside the TCB0 capture ISR every time a rising edge is detected
52   * by AC0 and routed through EVSYS to TCB0. A value of 0 means no valid
53   * measurement has been taken yet.
54   */
55  volatile uint32_t g_pwm_frequency_hz = 0;
56
57  /* ---------------------------------------------------------------------------
58   * Function prototypes
59   * --------------------------------------------------------------------------- */
60  static void clock_init(void);
61  static void tca0_pwm_init(void);
62  static void gpio_init(void);
63  static void ac0_init(void);
64  static void evsys_init(void);
65  static void tcb0_freq_measure_init(void);
66
67  /* ===========================================================================
68   * Peripheral initialisation functions
69   * =========================================================================== */
70
71  /**
72   * @brief Initialise the main clock to use the internal 4 MHz oscillator.
73   *
74   * The AVR128DA48 starts up with the internal 4 MHz oscillator selected by
75   * default, so no register writes are strictly required. This function is
76   * provided for clarity and to make future clock changes straightforward.
77   */
78  static void clock_init(void)
79  {
80      /* Default: internal 4 MHz oscillator, no prescaler. Nothing to change. */
```

14. Today Lab: Measuring Frequency

```
78  static void clock_init(void)
79  {
80      /* Default: internal 4 MHz oscillator, no prescaler. Nothing to change. */
81  }
82
83  /**
84   * @brief Configure GPIO pins used by TCA0, AC0, and the debug LED.
85   *
86   * Pin assignments:
87   *   - PA0 : TCA0 WO0 output (PWM). Set to output; peripheral override handled
88   *           by PORTMUX and TCA0 enable.
89   *   - PA7 : AC0 positive input AIN0. Set to input, disable digital input buffer
90   *           and pull-up to reduce noise on the analogue input.
91   *   - PC6 : On-board LED (active low on Curiosity Nano). Set to output, starts
92   *           high (LED off).
93   */
94  static void gpio_init(void)
95  {
96      /* PA0 – TCA0 WO0 PWM output */
97      PORTA.DIRSET = PIN0_bm;
98
99      /* PA7 – AC0 AIN0 positive input (analogue). Disable digital input buffer. */
100     PORTA.DIRCLR  = PIN7_bm;
101     PORTA.PIN7CTRL = PORT_ISC_INPUT_DISABLE_gc;
102
103     /* PC6 – on-board LED, output, default high (off) */
104     PORTC.DIRSET  = PIN6_bm;
105     PORTC.OUTSET  = PIN6_bm;
106 }
107
108 /**
109  * @brief Initialise TCA0 in single-slope PWM mode to produce a 1 kHz,
110  *        50 % duty cycle signal on WO0 (PA0).
111  *
112  * Timer clock  = F_CPU / DIV16 = 4 000 000 / 16 = 250 000 Hz
113  * PWM period   = (PER + 1) ticks = 250 ticks ? 1 kHz
114  * Duty cycle   = CMP0 / (PER + 1) = 125 / 250 = 50 %
115  *
116  * PORTMUX is configured to keep WO0 on PA0 (default alternate is not needed).
117  */
118 static void tca0_pwm_init(void)
119 {
```

14. Today Lab: Measuring Frequency

```
118 static void tca0_pwm_init(void)
119 {
120     /* Route WO0 to PA0 (default mapping – PORTMUX reset value is 0) */
121     PORTMUX.TCAROUTEA = PORTMUX_TCA0_PORTA_gc;
122 
123     /* Set period and compare registers */
124     TCA0.SINGLE.PER  = TCA0_PER_VALUE;
125     TCA0.SINGLE.CMP0 = TCA0_CMP0_VALUE;
126 
127     /* Single-slope PWM, enable WO0 compare output, DIV16 prescaler, enable */
128     TCA0.SINGLE.CTRLB = TCA_SINGLE_WGMODE_SINGLESLOPE_gc
129                       | TCA_SINGLE_CMP0EN_bm;
130 
131     TCA0.SINGLE.CTRLA = TCA0_PRESCALER_DIV
132                       | TCA_SINGLE_ENABLE_bm;
133 }
134 
135 /**
136  * @brief Initialise AC0 to compare AIN0 (PA7) against the internal DAC
137  *        voltage reference (set to VDD/2) and generate an event on its output.
138  *
139  * The PWM signal driven onto PA0 is wired externally to PA7 (AIN0).
140  * AC0 is configured to:
141  *   - Use AIN0 (PA7) as the positive input.
142  *   - Use the internal DAC output as the negative input (set to ~VDD/2 ? 1.65 V
143  *     at 3.3 V supply so that the AC output mirrors the digital PWM level).
144  *   - Enable the AC output to the Event System so that EVSYS can route it.
145  *
146  * @note The AC output event is a level signal. TCB0 in FRQMEAS mode uses the
147  *       rising edge of the event channel to trigger a capture, which corresponds
148  *       to the rising edge of the PWM signal.
149  */
150 static void ac0_init(void)
151 {
152     /* Set DAC reference: use VDD as the voltage reference */
153     VREF.ACREF = VREF_REFSEL_VDD_gc;
154 
155     /* DAC0: enable, output to AC0 negative input, value = 128 (? VDD/2) */
156     DAC0.CTRLA  = DAC_OUTEN_bm | DAC_ENABLE_bm;
157     DAC0.DATA   = 128U << DAC_DATA_gp; /* 8-bit left-aligned value */
158 
159     /* AC0 MUX: positive = AIN0 (PA7), negative = DAC */
```

14. Today Lab: Measuring Frequency

```
150 static void ac0_init(void)
151 {
152     /* Set DAC reference: use VDD as the voltage reference */
153     VREF.ACREF = VREF_REFSEL_VDD_gc;
154 
155     /* DAC0: enable, output to AC0 negative input, value = 128 (? VDD/2) */
156     DAC0.CTRLA  = DAC_OUTEN_bm | DAC_ENABLE_bm;
157     DAC0.DATA   = 128U << DAC_DATA_gp; /* 8-bit left-aligned value */
158 
159     /* AC0 MUX: positive = AIN0 (PA7), negative = DAC */
160     AC0.MUXCTRLA = AC_MUXPOS_AINP0_gc | AC_MUXNEG_DACREF_gc;
161 
162     /* Enable AC0, output to pin disabled, enable event output (OUTEN not
163      * needed for event routing – the AC output is automatically available
164      * as an event generator once the AC is enabled). */
165     AC0.CTRLA = AC_ENABLE_bm;
166 }
167 
168 /**
169  * @brief Configure EVSYS to route AC0 output to TCB0 event input.
170  *
171  * Channel 0 is used:
172  *   - Generator : AC0 output (EVSYS_CHANNEL0_AC0_OUT_gc)
173  *   - User      : TCB0 capture event input (EVSYS_USER_TCB0CAPT_CHANNEL0_gc)
174  */
175 static void evsys_init(void)
176 {
177     /* Assign AC0 output as generator on Event Channel 0 */
178     EVSYS.CHANNEL0 = EVSYS_CHANNEL0_AC0_OUT_gc;
179 
180     /* Connect TCB0 capture input to Event Channel 0 */
181     EVSYS.USERTCB0CAPT = EVSYS_USER_CHANNEL0_gc;
182 }
183 
184 /**
185  * @brief Initialise TCB0 in Frequency Measurement mode to capture the period
186  *        of rising edges delivered by AC0 through EVSYS.
187  *
188  * In FRQMEAS mode the counter runs freely and resets on each capture event
189  * (rising edge of the event signal). The captured value equals the number of
190  * TCB clock ticks between two consecutive rising edges, i.e. the PWM period.
191  *
192  * TCB clock = F_CPU / 2 = 2 000 000 Hz (DIV2 prescaler)
```

14. Today Lab: Measuring Frequency

```
184 /**
185  * @brief Initialise TCB0 in Frequency Measurement mode to capture the period
186  *        of rising edges delivered by AC0 through EVSYS.
187  *
188  * In FRQMEAS mode the counter runs freely and resets on each capture event
189  * (rising edge of the event signal). The captured value equals the number of
190  * TCB clock ticks between two consecutive rising edges, i.e. the PWM period.
191  *
192  * TCB clock = F_CPU / 2 = 2 000 000 Hz (DIV2 prescaler).
193  * Frequency  = TCB_CLOCK_HZ / captured_ticks.
194  *
195  * The capture interrupt is enabled so that the ISR can read CCMP and update
196  * the global frequency variable.
197  */
198 static void tcb0_freq_measure_init(void)
199 {
200     /* Select clock source: CLK_PER divided by 2 */
201     TCB0.CTRLA = TCB0_PRESCALER_DIV;
202
203     /* Frequency measurement mode, enable capture event input */
204     TCB0.CTRLB = TCB_CNTMODE_FRQ_gc;
205
206     /* Enable capture interrupt */
207     TCB0.INTCTRL = TCB_CAPT_bm;
208
209     /* Enable TCB0 */
210     TCB0.CTRLA |= TCB_ENABLE_bm;
211 }
212
213 /* ============================================================================
214  * Interrupt Service Routines
215  * ============================================================================ */
216
217 /**
218  * @brief TCB0 Capture ISR - computes PWM frequency from the captured period.
219  *
220  * In FRQMEAS mode, reading CCMP automatically clears the interrupt flag and
221  * resets the counter for the next measurement. The captured value is the
222  * number of TCB0 clock ticks that elapsed between the two most recent rising
223  * edges of the AC0 output event.
224  *
225  * Frequency (Hz) = TCB0_CLOCK_HZ / captured_ticks
226  *
```

14. Today Lab: Measuring Frequency

```
217 /**
218  * @brief TCB0 Capture ISR – computes PWM frequency from the captured period.
219  *
220  * In FRQMEAS mode, reading CCMP automatically clears the interrupt flag and
221  * resets the counter for the next measurement. The captured value is the
222  * number of TCB0 clock ticks that elapsed between the two most recent rising
223  * edges of the AC0 output event.
224  *
225  * Frequency (Hz) = TCB0_CLOCK_HZ / captured_ticks
226  *
227  * The result is stored in the global variable @ref g_pwm_frequency_hz.
228  * If the captured value is zero (should not occur in normal operation) the
229  * frequency is left unchanged to avoid a division-by-zero fault.
230  */
231 ISR(TCB0_INT_vect)
232 {
233     /* Reading CCMP clears the interrupt flag (hardware behaviour in FRQMEAS). */
234     uint16_t captured = TCB0.CCMP;
235 
236     if (captured != 0U)
237     {
238         g_pwm_frequency_hz = (uint32_t)TCB0_CLOCK_HZ / (uint32_t)captured;
239     }
240 }
241 
242 /* ============================================================================
243  * Main entry point
244  * ============================================================================ */
245 
246 /**
247  * @brief Application entry point.
248  *
249  * Initialise all peripherals in the required order and then enters an
250  * infinite super-loop. The frequency measurement is performed entirely in the
251  * TCB0 capture ISR; the super-loop can be extended to use @ref g_pwm_frequency_hz
252  * for display, communication, or control purposes.
253  *
254  * Initialisation order:
255  *  1. Clock  – confirm 4 MHz internal oscillator.
256  *  2. GPIO   – configure pins before enabling peripherals.
257  *  3. TCA0   – start PWM generation on PA0.
258  *  4. AC0    – enable analogue comparator (after GPIO so AIN0 buffer is off).
259  *  5. EVSYS  – connect AC0 → TCB0 before enabling TCB0.
```

14. Today Lab: Measuring Frequency

```
255  *  1. Clock  - confirm 4 MHz internal oscillator.
256  *  2. GPIO   - configure pins before enabling peripherals.
257  *  3. TCA0   - start PWM generation on PA0.
258  *  4. AC0    - enable analogue comparator (after GPIO so AIN0 buffer is off).
259  *  5. EVSYS  - connect AC0 ? TCB0 before enabling TCB0.
260  *  6. TCB0   - start frequency measurement, enable capture interrupt.
261  *  7. sei()  - enable global interrupts.
262  *
263  * @return This function never returns (embedded super-loop).
264  */
265 int main(void)
266 {
267     // Initialise the main clock
268     clock_init();
269     // Configure GPIO pins used
270     gpio_init();
271     // Initialise TCA0 to generate a PWM signal
272     tca0_pwm_init();
273     // Initialise AC0 to compare
274     ac0_init();
275     // Configure EVSYS to route AC0 output to TCB0 event input
276     evsys_init();
277     // Initialise TCB0 in Frequency Measurement mode
278     tcb0_freq_measure_init();
279     // Enable global interrupts
280     sei();
281
282     while (1)
283     {
284         /*
285          * g_pwm_frequency_hz is updated by the TCB0 ISR.
286          * Add application logic here (e.g. USART output, LED indication).
287          *
288          * Example: toggle LED if measured frequency is within ±5 % of 1 kHz.
289          *   if (g_pwm_frequency_hz >= 950 && g_pwm_frequency_hz <= 1050)
290          *       PORTC.OUTTGL = PIN6_bm;
291          */
292     }
293
294     return 0; /* Never reached */
295 }
```

15. What You Learned Today

15. What You Learned Today

You now understand how to:

- Use the analog comparator as a fast decision block
- Stabilize decisions using hysteresis
- Route comparator output through the Event System
- Trigger peripherals without CPU work
- Build more deterministic, lower-power firmware architectures

16. What Comes Next

16. What Comes Next

Day 18 - ADC: Practical Sampling, Averaging, and Hardware Triggers

Next you will connect the dots:

- ADC triggered by timers/events
- Sampling strategies that do not block
- Averaging, oversampling, noise reduction
- Real sensor workflows (not just reading a number)

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course